

Proposal: Web Crawl, Accessibility & Compliance Monitoring System

Overview:

The Web Crawl, Accessibility & Compliance Monitoring System is an internal enterprise platform designed to centrally manage and monitor multiple organizational websites and web pages. The system enables teams to trigger on-demand website scans that detect broken links, accessibility violations (WCAG), and compliance issues. It provides structured issue tracking, ownership assignment, and real-time dashboards so organizations can proactively identify problems instead of relying on customer complaints, spreadsheets, or manual communication.

Key Features:

Website Registration & Monitoring Scope

- Register and manage multiple domains and sub-sites from a centralized interface
- Define crawl scope including full site, selected paths, and exclusions
- Maintain lifecycle tracking of pages (discovered, changed, removed)

Automated Crawling & Page Discovery

- On-demand or scheduled crawling initiated by users or system rules
- Automatically discover internal links and assets across the monitored domain
- Track page versions and detect content changes between scans

Accessibility & Compliance Analysis

- Evaluate pages against accessibility standards (WCAG) using automated analysis
- Detect compliance issues such as missing disclosures, broken policies, and outdated content
- Record structured findings linked to specific page versions for traceability

Issue Management & Workflow Tracking

- Create actionable findings with states (Open, Acknowledged, Resolved, Ignored)
- Allow manual notes, comments, and ownership assignment
- Maintain remediation history for each issue and verify fixes through re-scanning

Reporting, Trends & Audit Export

- Real-time dashboard showing risk exposure and system status
- Historical trend analysis of accessibility and compliance performance
- Generate exportable reports suitable for internal audits or regulatory review

Non-Functional requirements

- **Scalability:** Support crawling and analysing thousands to millions of pages
- **Performance:** Web crawling and analysis must not block user interactions
- **Consistency:** Findings must reliably map back to the correct page versions
- **Fault tolerance:** Crawl failures should not halt the entire system
- **Observability:** Visibility into crawl progress, failures, and processing delays
- **Security:** Controlled access to reports, findings, and administrative actions
- **Asynchronous processing:** All crawling and analysis must run in background processes

Technical Specifications:

- Frontend: React.js with TypeScript
- Backend: Node.js with Express.js (Microservices architecture)
- Database: PostgreSQL
- APIs: RESTful APIs with JWT authentication
- Caching: Redis
- Web Crawling: Puppeteer
- Job Queue: RabbitMQ
- WCAG accessibility analysis: axe-core

Ballpark Estimate:

Total duration: 4.5 – 5 weeks

Test Estimate doc:

<https://docs.google.com/spreadsheets/d/1mYqTiN6B89p24GAOgK1zSAUs6TTAdWSjg-V10i16Xrs/edit?usp=sharing>

Assumptions

The following assumptions are made for the system design and estimates:

- The platform will operate as an internal enterprise service.
- Target websites allow automated crawling and are not blocked by bot protection mechanisms.
- Websites are renderable via headless browser (Puppeteer).
- Accessibility compliance evaluation is limited to automated WCAG checks.
- Crawling frequency will initially be moderate (e.g., daily or weekly scans), not real-time monitoring.

Risks & Technical Challenges

1. Crawl Blocking / Bot Protection:

Some websites may use CAPTCHA, rate-limiting, or bot protection services which can prevent headless browsers from accessing pages. This may reduce scan coverage and require crawl throttling or allow-listing.

2. Dynamic Frontend Rendering:

Modern websites using heavy client-side frameworks may load content asynchronously. Incorrect wait strategies may lead to incomplete DOM analysis and false accessibility results.

3. Large Site Scale:

Very large websites (thousands of pages) can significantly increase crawl duration and worker resource usage. Improper queue handling could create backlog or memory pressure.

4. Non-Deterministic Page Content:

Pages containing frequently changing data (timestamps, rotating banners, ads) may produce new page hashes each crawl, causing unnecessary re-analysis and inflated issue counts.

5. Worker Resource Consumption:

Headless browsers are CPU and memory intensive. Poor concurrency control may crash worker nodes or degrade system performance.

6. Accessibility False Positives:

Automated tools cannot detect all accessibility issues and may occasionally produce false positives. The system must allow acknowledgement and manual review.

MVP Phase Estimate

The MVP (Minimum Viable Product) focuses on delivering a functional end-to-end monitoring pipeline capable of crawling websites, detecting accessibility violations, storing historical findings, and generating basic operational reports.

MVP Capabilities

The initial release will include:

- Registration of websites and definition of crawl scope
- Manual triggering of site scans
- Distributed crawling and page discovery
- Accessibility scanning using automated rule evaluation
- Storage of page versions and issue findings
- Issue lifecycle states (Open, Acknowledged, Resolved, Ignored)
- Audit history of issue status changes
- Basic dashboard with issue counts and accessibility score
- Daily aggregated metrics for trend visualization
- Exportable compliance report (CSV or PDF)

Excluded from MVP

The following items are intentionally excluded:

- Real-time notifications (email)
- Performance and SEO analysis
- Advanced workflow management or ticketing integration
- Custom rule authoring UI

Estimated Timeline (MVP)

Phase	Duration
Project Setup	1 day
Database Design	1 day
Crawler and Page discovery Workers	2 – 3 days
Accessibility Analysis Integration (axe-core)	3 – 5 days
Issue tracking and status history	2 – 3 days

Dashboard and UI Design	1 – 1.5 weeks
-------------------------	----------------------

Estimated MVP Duration: 4–5 weeks